ACS MILESTONE 22

User Guide

This guide explains how to run the current application, use the classic ACS visual mode, play the sample adventure, save progress, use the editor, validate drafts, and move from a local draft to a published release.

What is included right now

- The playable runtime at `apps/web/index.html`
- The browser editor at `apps/web/editor.html`
- The local project, validation, and release API at `apps/api/dist/index.js`

Starting The Application

Build the workspace if needed, then start both servers from the repo root.

Web Server

```
node .\apps\web\server.mjs
```

Default URL:

```
http://localhost:4173/
```

Common alternate URL in this environment:

```
http://localhost:4317/
```

API Server

```
node .\apps\api\dist\index.js
```

Local API check:

```
http://localhost:4318/api/session
```

Editor URL

```
http://localhost:4317/apps/web/editor.html
```

Playing The Game

The runtime can load the built-in sample adventure, a local draft playtest, or a published release.

Sample objective

1. Speak to the Oracle.
2. Enter the shrine.
3. Claim the Solar Seal.
4. Return to the Oracle.

Keyboard controls

- `WASD` or arrow keys move
- `E` interacts
- `Q` inspects
- `Enter`, `Space`, or `E` advances dialogue
- Arrow keys scroll longer dialogue while Classic ACS dialogue is active

Runtime panels

- Map and player position
- Session source
- Objective panel
- Save controls
- State summary and event log

Turn-based enemy timing

Successful player actions advance the turn count, but blocked movement does not give enemies a free action. Enemy behavior can define a `turnInterval`; the sample Shrine Wolf acts every third successful player turn, giving you room to maneuver instead of shadowing every step.

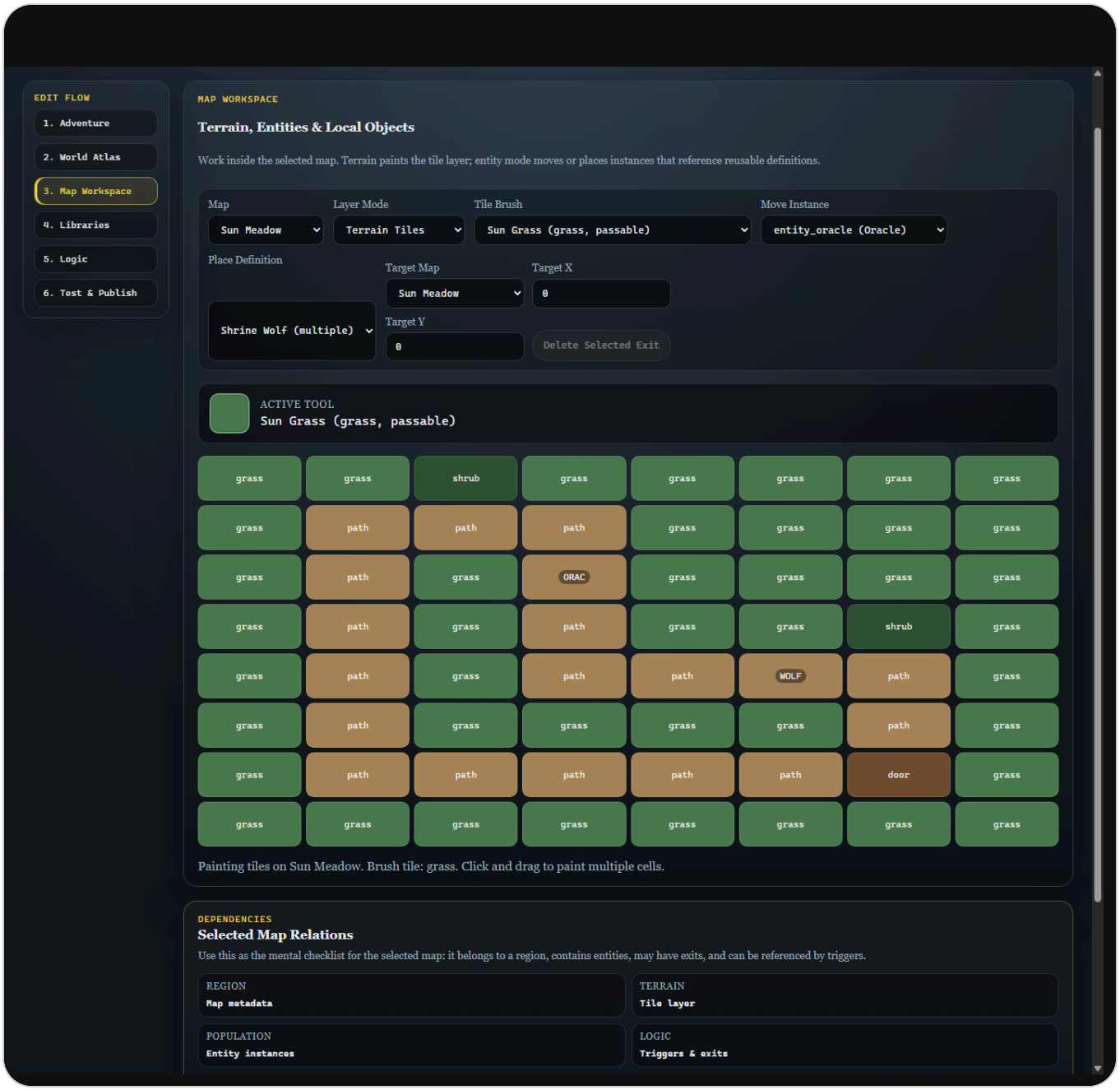
Saving And Loading Progress

The runtime uses separate local save slots for the sample adventure, draft playtests, and published releases.

- **Save** stores the current runtime snapshot in IndexedDB.
- **Load** restores the most recent snapshot for the current source. The runtime starts fresh from the adventure start state and does not automatically restore saves on page load.
- **Reset** restarts the live session without deleting the saved snapshot.

Using The Editor

Open the editor at `http://localhost:4317/apps/web/editor.html`.



The current editor supports metadata editing, tile painting with a persistent brush, reusable tile definition editing, quest definition editing, adding and repositioning entity instances, reusable entity definition editing, dialogue text editing, structured trigger editing, map structure editing, blank map creation, shared validation summaries, API-side `Validate Draft` checks, local draft saves, backend project saves, publishing releases, and opening those releases in the runtime.

Top buttons

- `Back To Play`
- `Save Draft`
- `Reset Draft`
- `Playtest Draft`

Toolbar controls

- `World Atlas` chooses the current map because maps belong to the region/adventure hierarchy
- `Layer Mode` switches between Terrain Tiles and Entity Instances
- `Tile` picks the tile to paint
- `Move Entity` picks the existing entity to move
- `Add Definition` chooses the reusable entity definition to place
- `Place New` switches entity mode into new-instance placement
- `Active Brush` shows the tile brush or entity placement target

Organized Edit Game Screen

The editor is now organized around the way game information relates: Adventure, World Atlas, Map Workspace, Libraries, Logic, and Test & Publish.

- `Edit Flow` navigation moves through the major authoring areas.
- `World Atlas` contains region/map thinking: selected map, category, region assignment, and blank-map creation.
- `Map Workspace` contains grid-centered work for terrain tiles and entity instances.
- `Dependencies` shows the selected map's relationship checklist: region, terrain, population, logic, and exits.
- `Libraries` contains reusable definitions and dialogue records.
- `Logic & Quests` contains trigger editing shaped around `When / If / Then`.
- `Test & Publish` contains validation, project saving, publishing, and release opening.

Tile Editing

1. Choose `Tiles` mode.
2. Select a tile id from the dropdown.
3. Check the `Active Brush` preview to confirm the selected tile.
4. Click a cell in the grid to paint a single tile, or click and drag to paint across multiple cells.

The selected tile stays loaded like a brush until you choose a different tile.

Tile Definition Editing

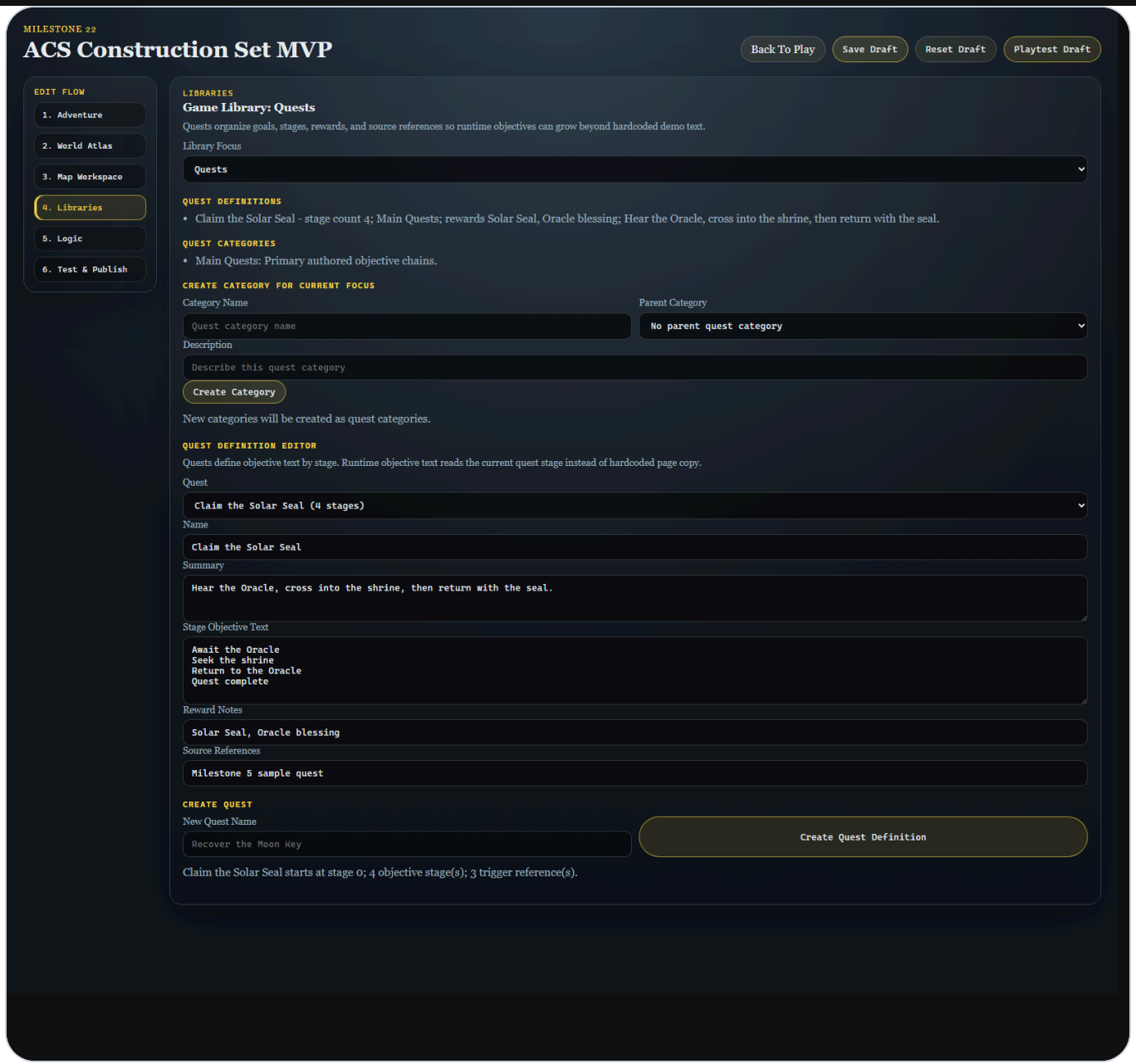
Tiles are now library objects, not just visual strings in a map layer. In `Libraries`, choose `Tiles` from `Library Focus` to edit reusable terrain definitions.

Milestone 21 tile fields

- `Name` is the designer-facing label.
- `Passability` can be `passable`, `blocked`, or `conditional`.
- `Description` and `Interaction Hint` explain what the tile represents.
- `Tags` organize terrain such as barriers, relics, water, or sci-fi objects.
- `Classic Sprite` maps the logical tile to the current Classic ACS presentation without hardcoding future HD or 3D art.

Quest Definition Editing

Milestone 22 makes quests first-class library objects instead of hardcoded objective text. In `Libraries`, choose `Quests` from `Library Focus` to edit objective chains.



Milestone 22 quest fields

- `Name` is the player/designer-facing quest title.
- `Summary` describes the durable premise.
- `Stage Objective Text` stores one objective line per stage.
- `Reward Notes` and `Source References` capture payoff and inspiration notes.
- The runtime Objective panel reads the current stage from `state.questStages`.

Entity Editing

Entity mode supports both moving existing instances and placing new instances from reusable definitions.

1. Choose `Entities` mode.

2. To move an instance, pick it from `Move Entity` and click a destination cell.
3. To add a new instance, choose a definition from `Add Definition`, click `Place New`, then click a destination cell.

Placement rules

- `singleton` definitions can appear once in the adventure. The Oracle is singleton.
- `multiple` definitions can be placed repeatedly. The Shrine Wolf is multiple.
- Validation reports a blocking error if a singleton definition has more than one instance.

Editing World Structure And Creating Maps

The `World Structure` panel edits the selected map name, category, and region assignment. The `Create Map` controls create a new blank map with a chosen category, optional region, width, height, and fill tile. Milestone 14 does not yet create exits automatically; map wiring remains future work.

Editing Dialogue And Triggers

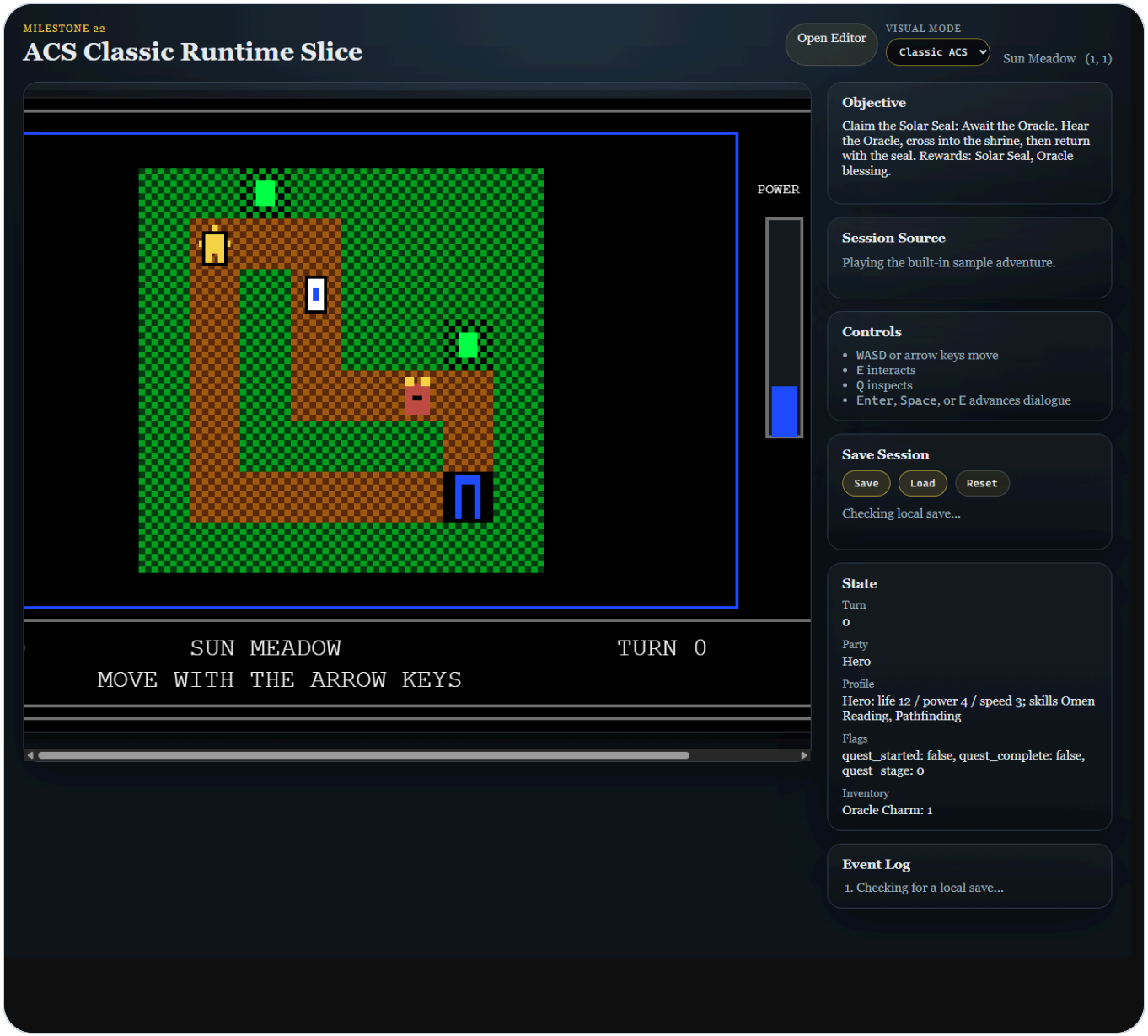
The `Dialogue` Library Focus edits existing dialogue records. The `Rule Trigger` panel edits existing trigger records, including type, optional map/x/y location, run-once behavior, conditions JSON, and actions JSON. These JSON areas are structured package data, not executable scripts; invalid JSON is rejected until corrected.

Validation

The validation panel runs the shared validation package and shows an error-and-warning summary plus a detailed issue list for the current draft. Use `Validate Draft` in the `Project & Release` panel when you want the local API to run the same validation report the publish flow uses. Blocking errors keep project save and publish actions disabled until they are fixed.

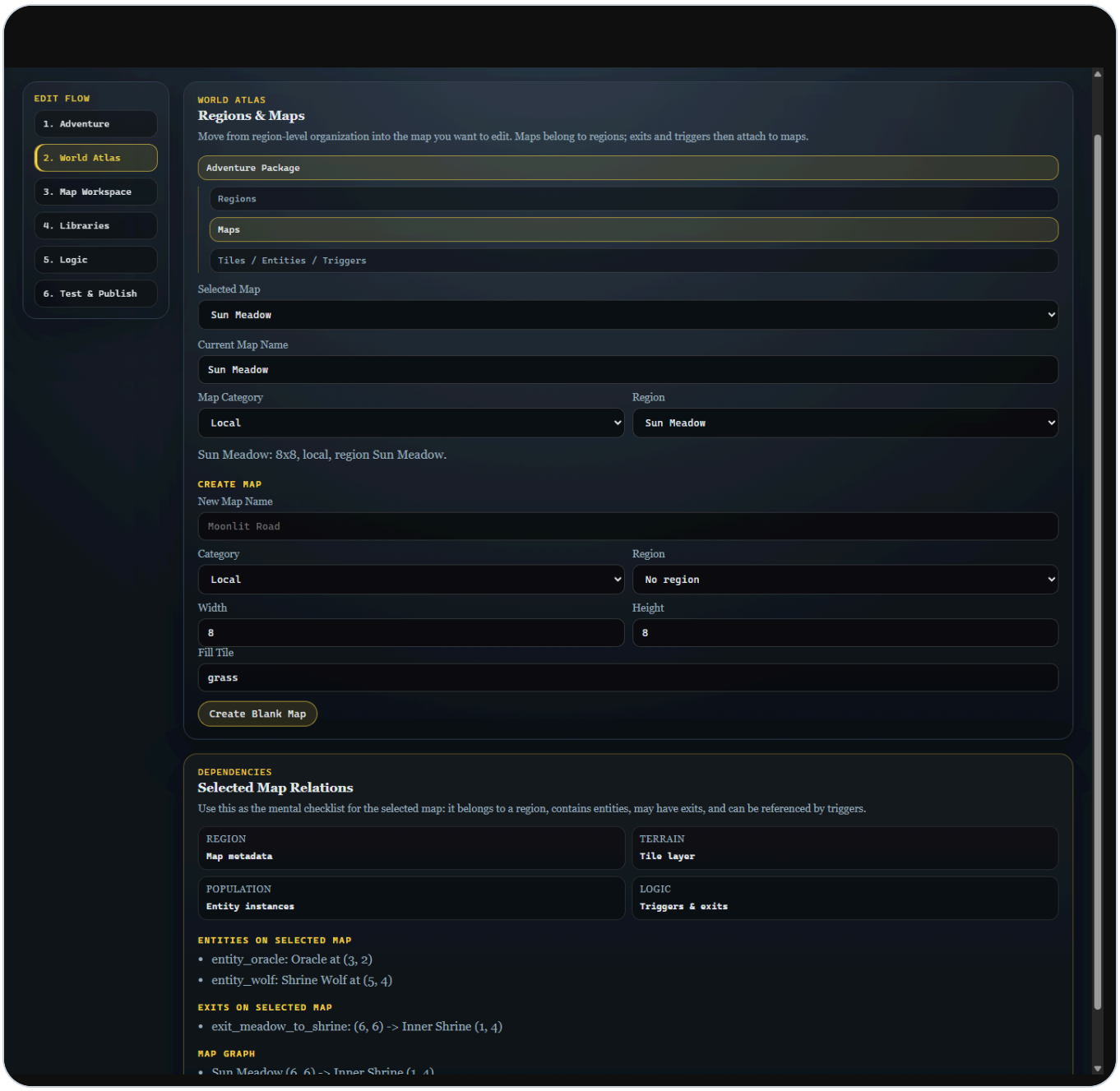
Tutorial: Try Every Current Feature

This walkthrough is the recommended smoke test after each milestone. It exercises every major feature currently available, highlights the newest Milestone 22 quest/objective workflow, and shows how tile, entity, dialogue, and trigger edits can combine into a miniature quest scene.



Goal of this tutorial

You will play the sample adventure, switch visual modes, test save/load/reset, edit metadata, use the organized Edit Flow screen, create a map, paint tiles, author exits, review the map graph, move and place entities, edit definitions, edit dialogue, edit triggers, create and place trigger markers, validate, save, publish, and open a release.



1. Start Both Servers

```
node .\apps\web\server.mjs
```

```
node .\apps\api\dist\index.js
```

Open the runtime at `http://localhost:4317/apps/web/index.html` and the editor at `http://localhost:4317/apps/web/editor.html`. The runtime is the player-facing game screen; the editor is the construction set; the API enables validation, project saving, publishing, and release loading.

2. Play The Sample Adventure First

1. Confirm `Visual Mode` is set to `Classic ACS`.
2. Move with `WASD` or the arrow keys.
3. Walk next to the Oracle and press `E` to interact.
4. Press `Enter`, `Space`, or `E` to advance dialogue without leaving the keyboard.
5. Press `Q` near the Oracle or on a tile to inspect.
6. Use the door tile to enter the Inner Shrine.
7. Move to the altar to claim the Solar Seal.
8. Return to the Oracle and interact again.

Watch the side panels and event log. Movement changes position and turn count. Classic ACS dialogue appears in the bottom message band and can be scrolled with arrow keys when it wraps; Debug Grid uses a separate dialogue panel. The altar trigger changes flags, inventory, and tile state. Enemy turns advance only on their configured cadence.

> **Clever observation:** the shrine is not just a painted room. It is a map with terrain, an altar tile, a reward trigger, dialogue, inventory changes, and enemy behavior layered together.

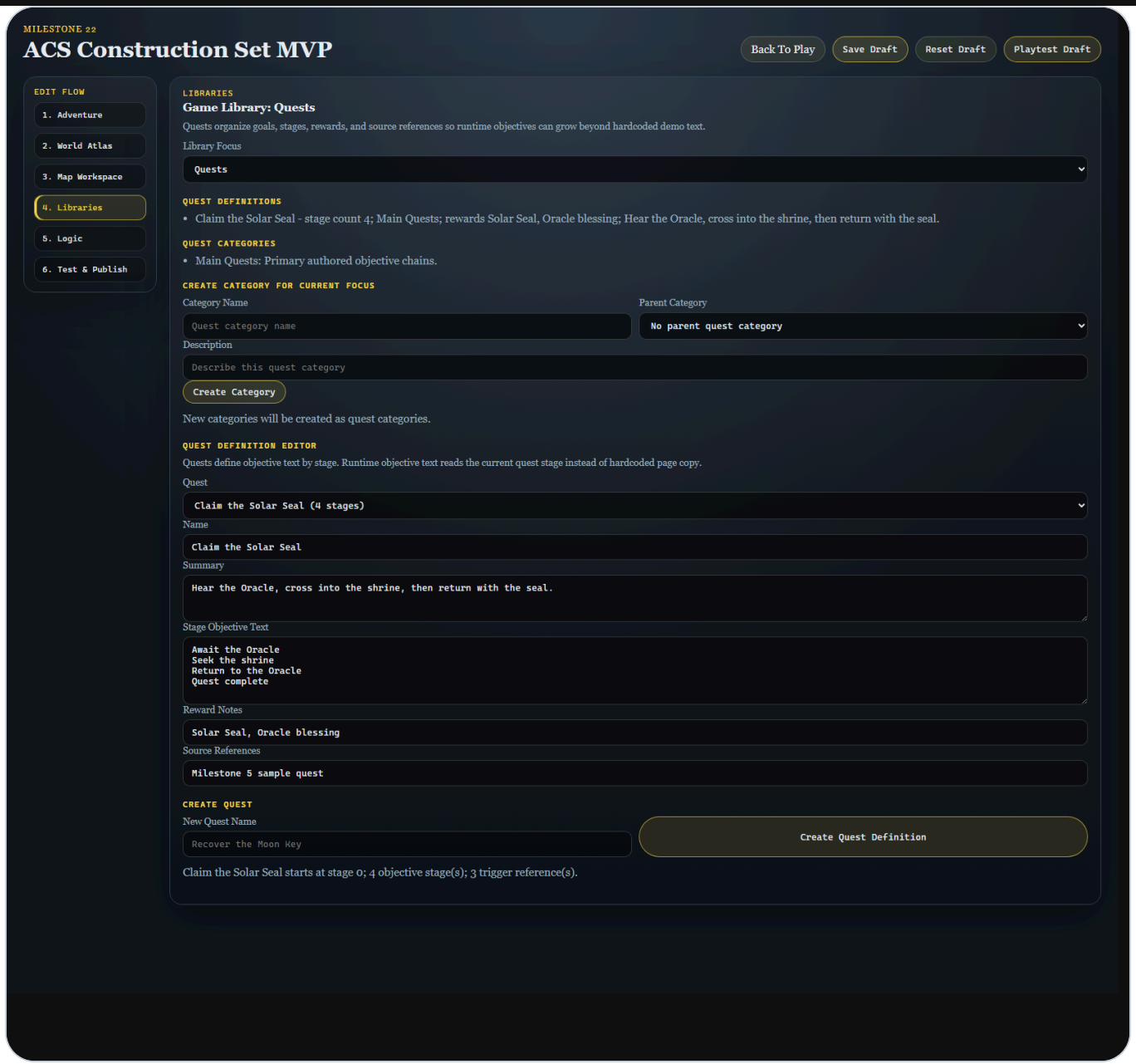
3. Try Runtime Save, Load, Reset, And Visual Mode

1. Click `Save` after moving a few steps.
2. Move somewhere else, then click `Load` and confirm the saved position is restored.
3. Click `Reset`, then click `Load` again to confirm the saved snapshot still exists.
4. Switch from `Classic ACS` to `Debug Grid`, then back to `Classic ACS`.

This proves visual mode is presentation-only and persistence stores a runtime snapshot rather than a second copy of map data.

Latest Milestone Highlight: Tile Definitions And Terrain Behavior

Milestone 21 makes terrain behavior reusable library data. A painted cell still stores a tile id, but that id now resolves to a `TileDefinition` with passability, hints, tags, categories, and classic sprite mapping. The editor authors the definition, validation checks references, runtime-core blocks movement into blocked terrain, and runtime-2d uses the classic sprite mapping for the current presentation mode.

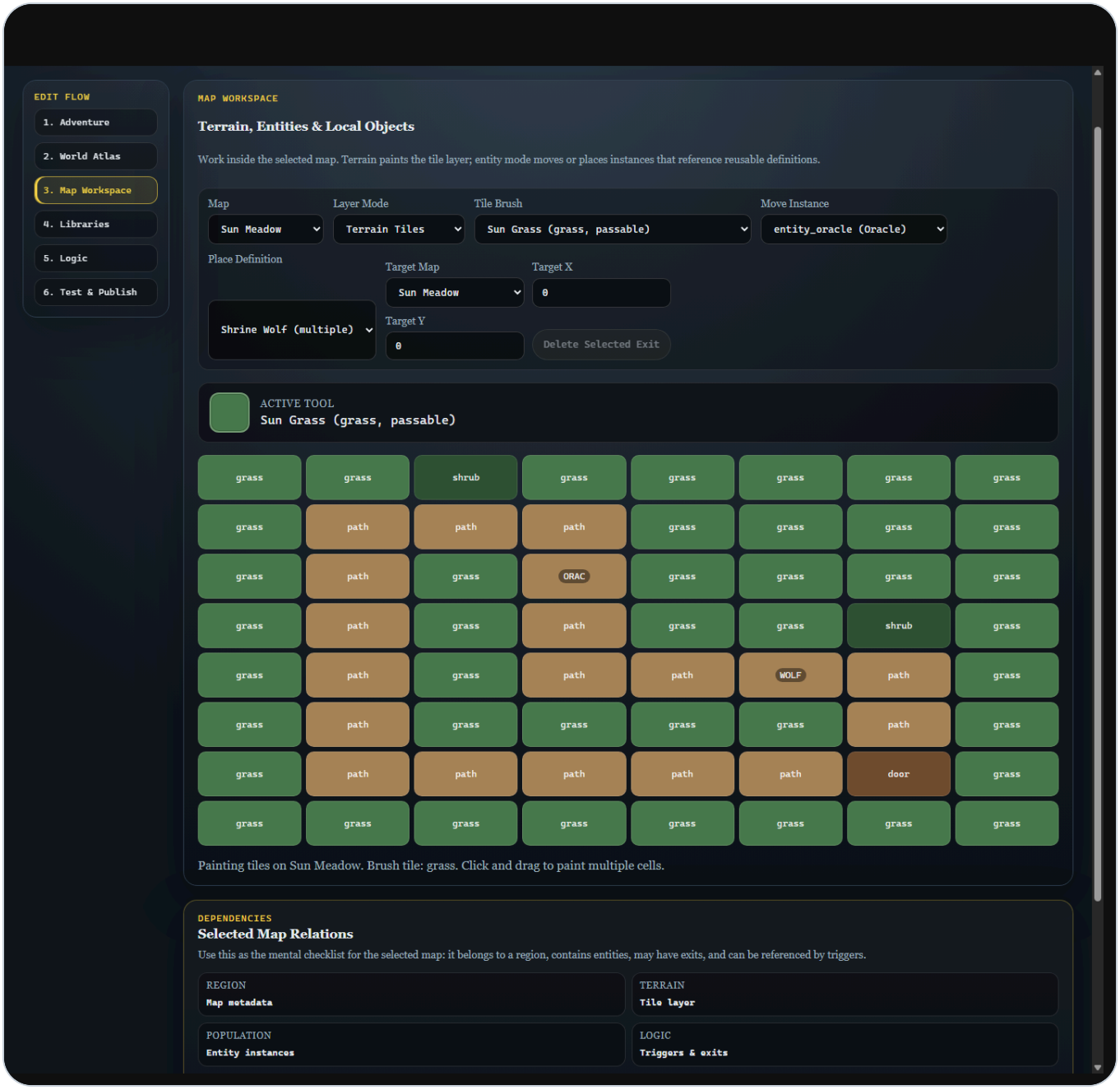


1. Open `Libraries` and set `Library Focus` to `Tiles`.
2. Select `Water`, `Shrub`, or `Stone` and confirm `Passability` is `blocked`.
3. Create a new tile named `Force Field`.
4. Set `Passability` to `blocked`, add a hint such as `The humming wall rejects your hand.`, and tag it `barrier`, `sci-fi`, and `energy`.
5. Set `Classic Sprite` to an existing sprite such as `water` until richer art packs arrive.
6. Return to `Map Workspace`, paint the Force Field as a barrier, then `Playtest Draft` and confirm the player cannot walk through it.

Adventuria-style idea: use the same blocked passability for a fantasy sacred flame, a sci-fi force field, an urban locked turnstile, or a mythic river. The behavior is shared data; the story and renderer make it feel different.

Previous Milestone Highlight: Exits, Portals, And Map Graphs

Milestone 20 made map-to-map travel an authored construction-set feature. Exits live on maps as data, the editor can create or update them from the grid, and the runtime consumes the same exit records during movement. That keeps the engine independent from the visual mode: a classic door tile, a sci-fi teleporter pad, an urban subway hatch, and a castle gate can all be the same logical exit with different presentation.

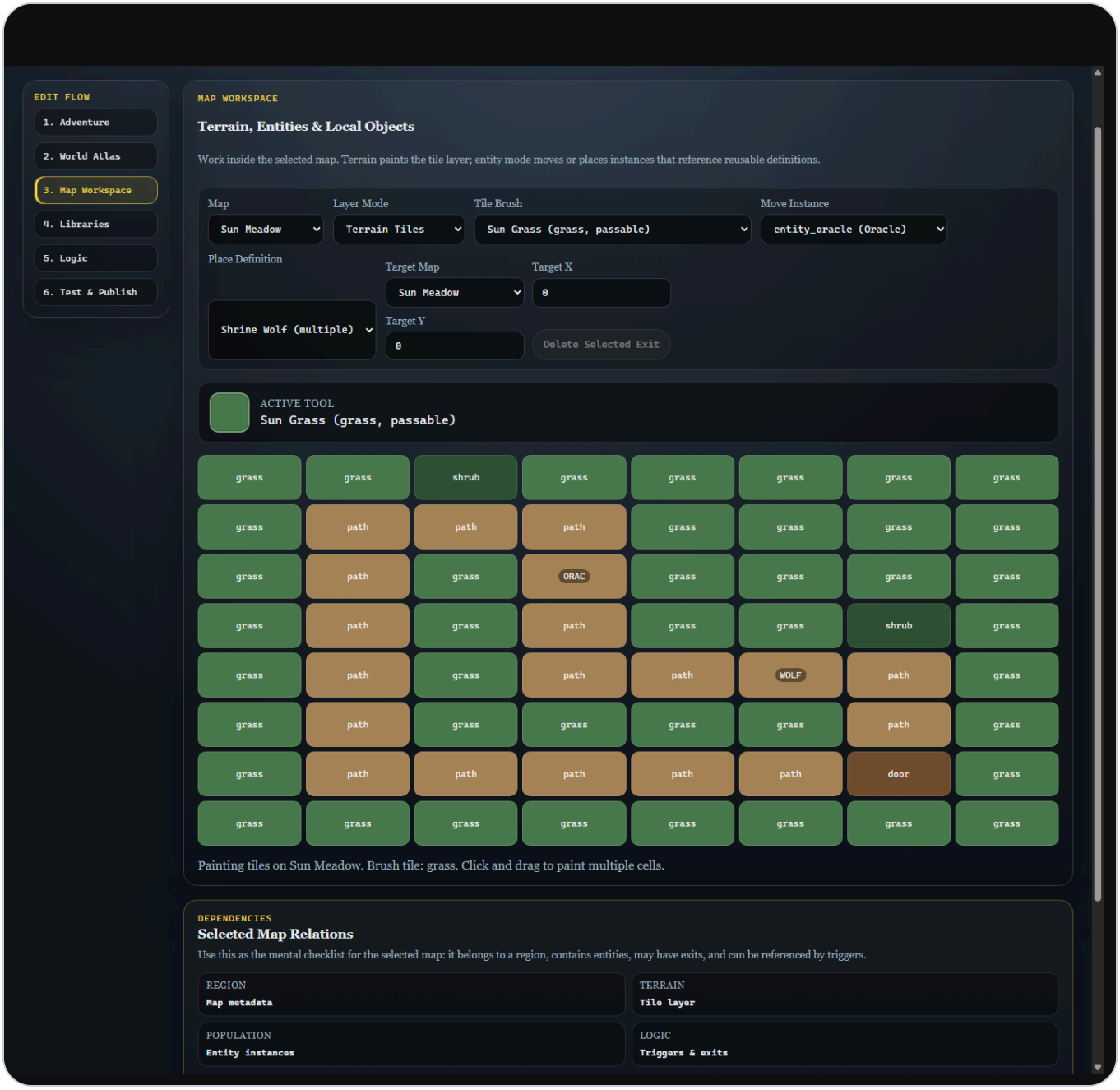


1. Open `Map Workspace`.
2. Set `Layer Mode` to `Exits & Portals`.
3. Choose `Inner Shrine` as the `Target Map`, with `Target X` set to `1` and `Target Y` set to `4`.
4. Click the Sun Meadow door cell at `(6, 6)` to inspect the existing shrine exit, or click another cell to create a new portal.
5. Read `Exits On Selected Map` and `Map Graph` in the Dependencies panel to confirm the outgoing link and the return link.

Adventuria-style idea: build one sampler world where a forest shrine door leads to fantasy myth, a glowing floor pad leads to a starship deck, a stairwell leads to an urban subway, and a castle gate leads to a siege map. The exits are just graph edges; the tile art and story make each edge feel different.

Previous Milestone Highlight: Map Context And Classified Libraries

Milestone 19 made the focused editor workflow more self-contained. You no longer have to return to `World Atlas` just to change the map being edited: `Map Workspace` now has its own `Map` selector, and `Logic` has a `Current Map Context` selector.



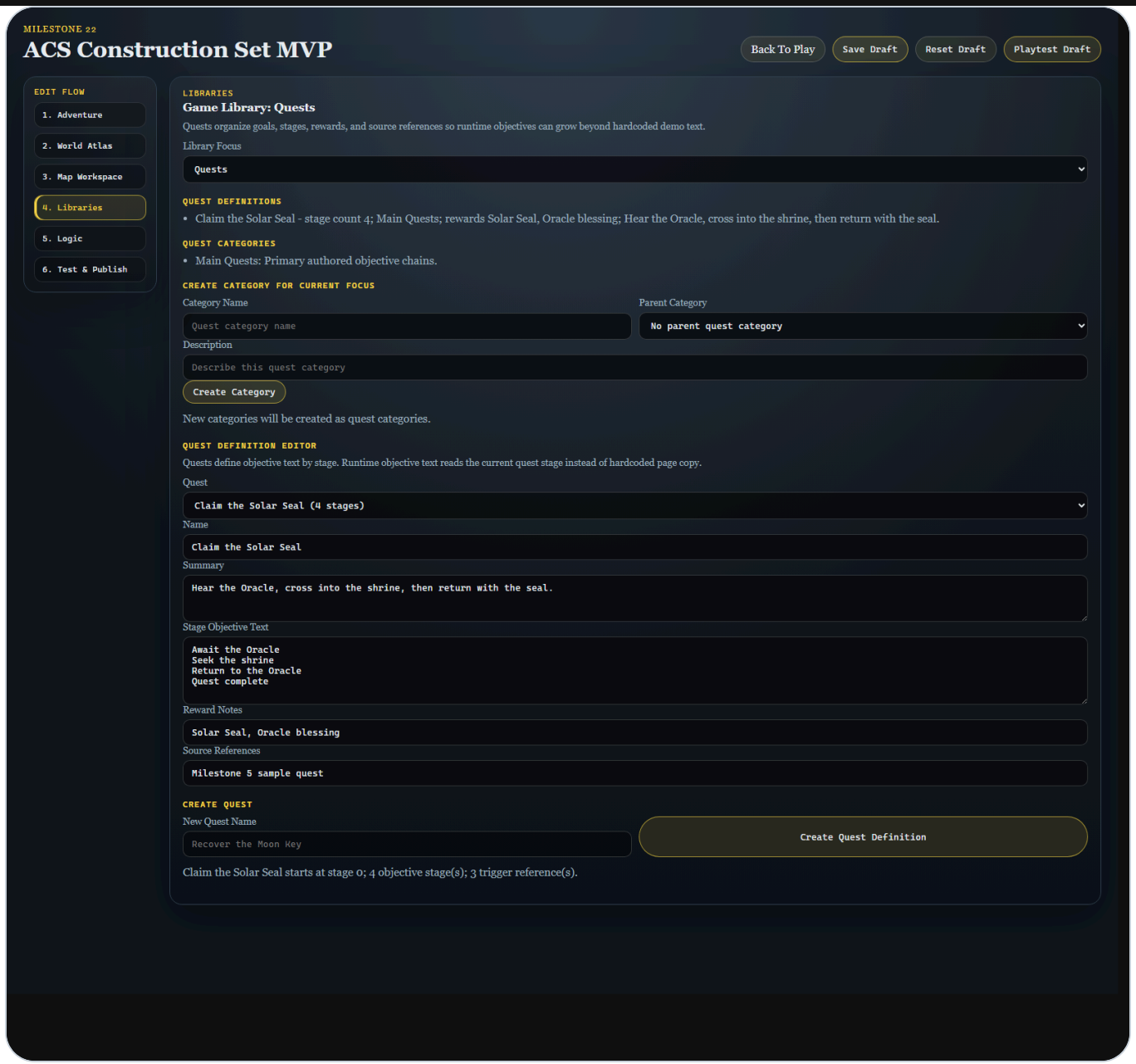
The `Libraries` panel now has a `Library Focus` selector. This is the first visible layer of the new organizing principle for reusable game-wide objects.

- `Entities`: reusable NPC, enemy, container, and player templates.
- `Items`: possessable objects such as relics, quest items, future weapons, treasure, tools, and consumables.
- `Skills`: reusable capabilities that entity profiles reference by ID instead of typed strings.
- `Dialogue`: reusable conversation/narration records that triggers and interactions reference by id.
- `Flags`: named state variables that trigger conditions/actions select from instead of freeform flag names.
- `Quests`: authored objective chains that can be classified and referenced by triggers.
- `Traits`, `Spells`, `Tiles`, `Assets`, and `Custom`: future-facing object classes that can already receive categories.

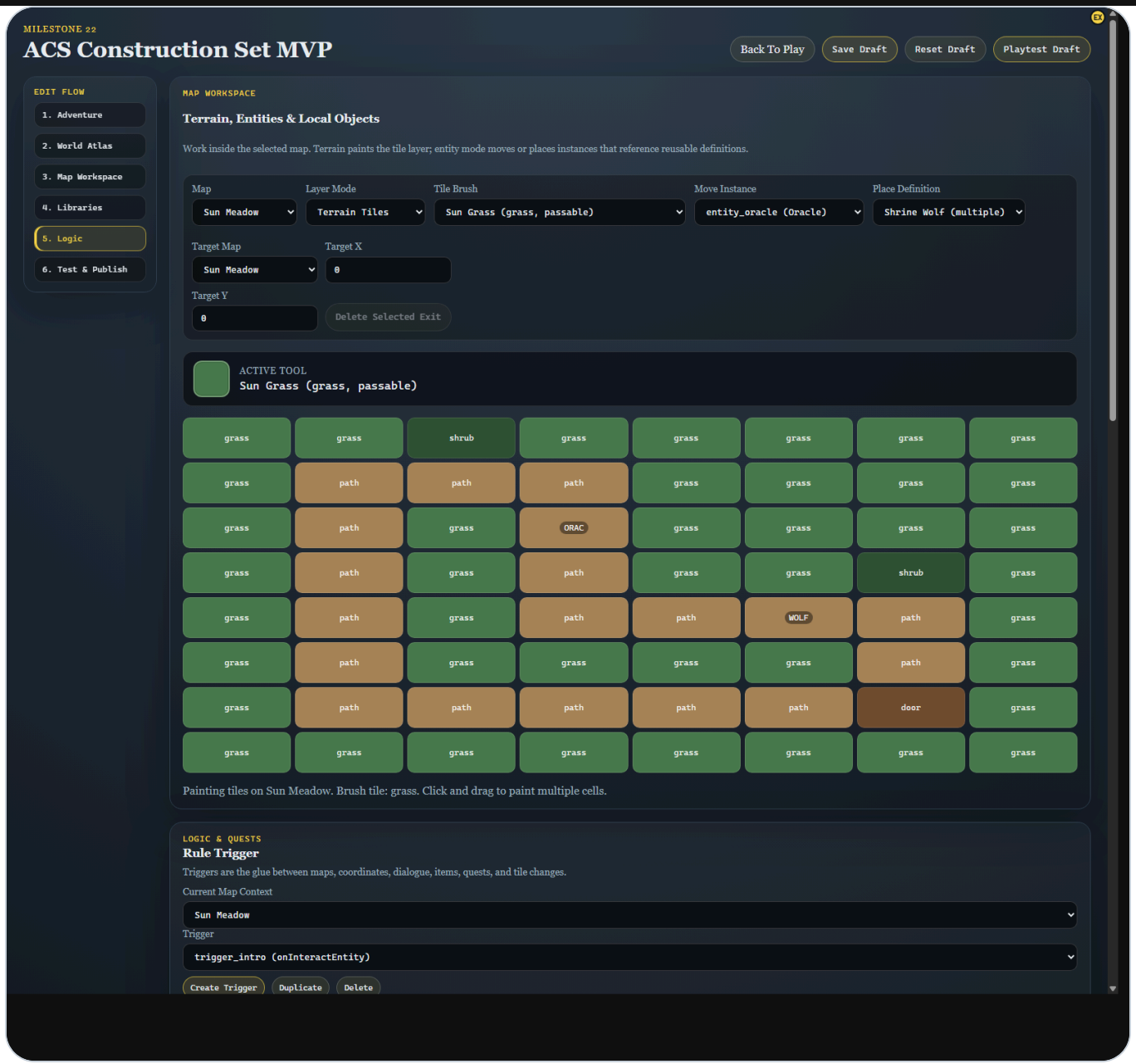
Try this in the tutorial: open `Libraries`, switch `Library Focus` between `Entities`, `Items`, `Skills`, `Dialogue`, `Flags`, `Quests`, and the future-facing library kinds. The title, help text, focused object list, category list, category creator, and editor visibility should change with the selected focus. Under `Items`, confirm `Solar Seal` appears as an item definition; under `Dialogue`, confirm the Oracle and shrine dialogue records appear as categorized reusable objects.

Latest Milestone Highlight: Quest Definitions And Runtime Objectives

Milestone 22 turns the side-panel objective into authored quest data. Quest definitions now have stage text, reward notes, source references, categories, editor creation/update controls, trigger-stage wiring, runtime `questStageSet` events, and validation for missing or out-of-range quest references.



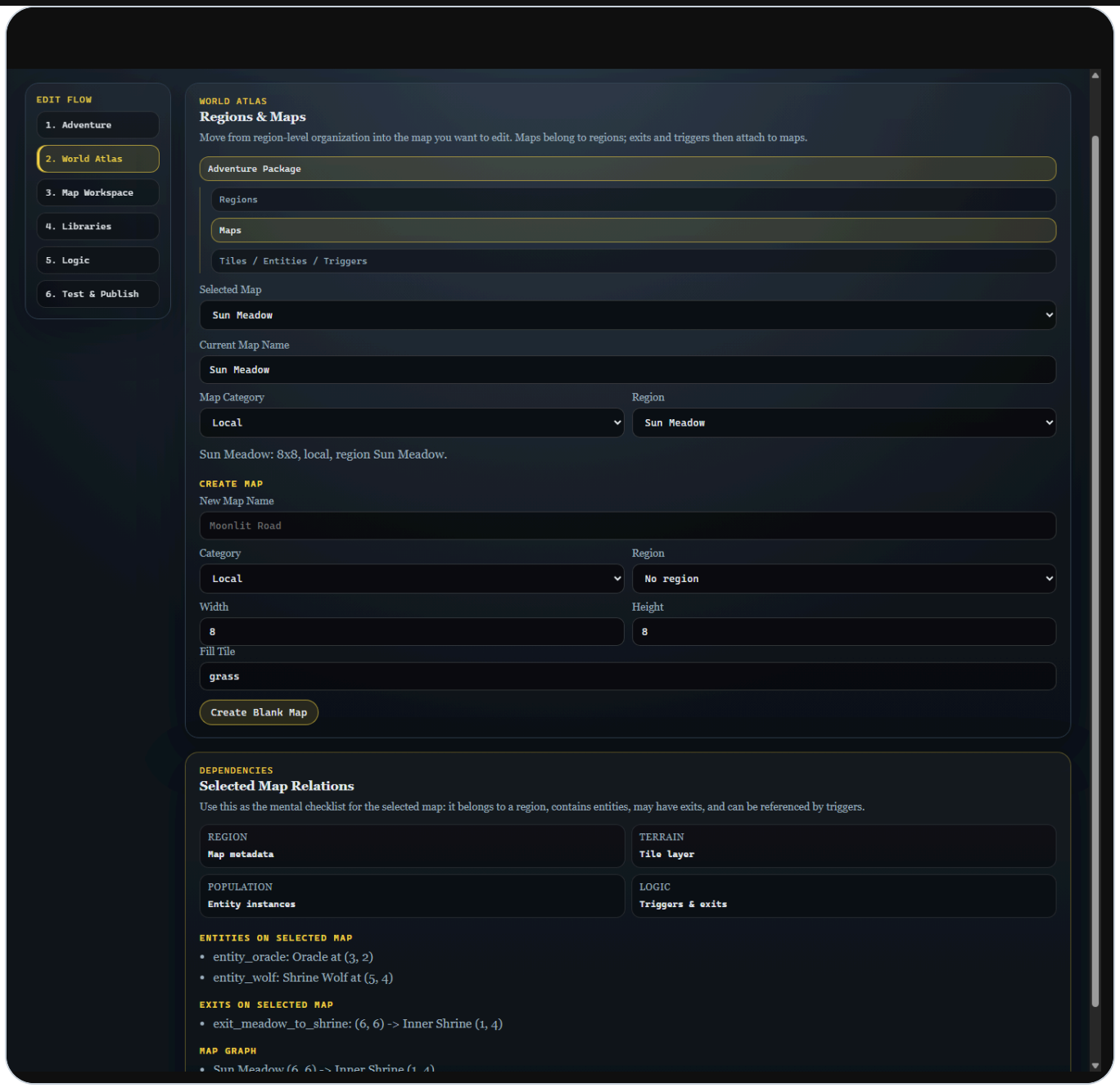
1. Open `Libraries` and set `Library Focus` to `Quests`.
2. Select `Claim the Solar Seal` and read the four objective stage lines.
3. Add a reward note such as `Moon Key` or `Safehouse Access`.
4. Create a new quest definition named `Recover the Moon Key` for a sci-fi, urban, or castle beat.
5. Open `Logic`, select a trigger, and add or inspect a `Set Quest Stage` action.
6. Playtest the draft and watch the Objective panel change as triggers advance the quest.



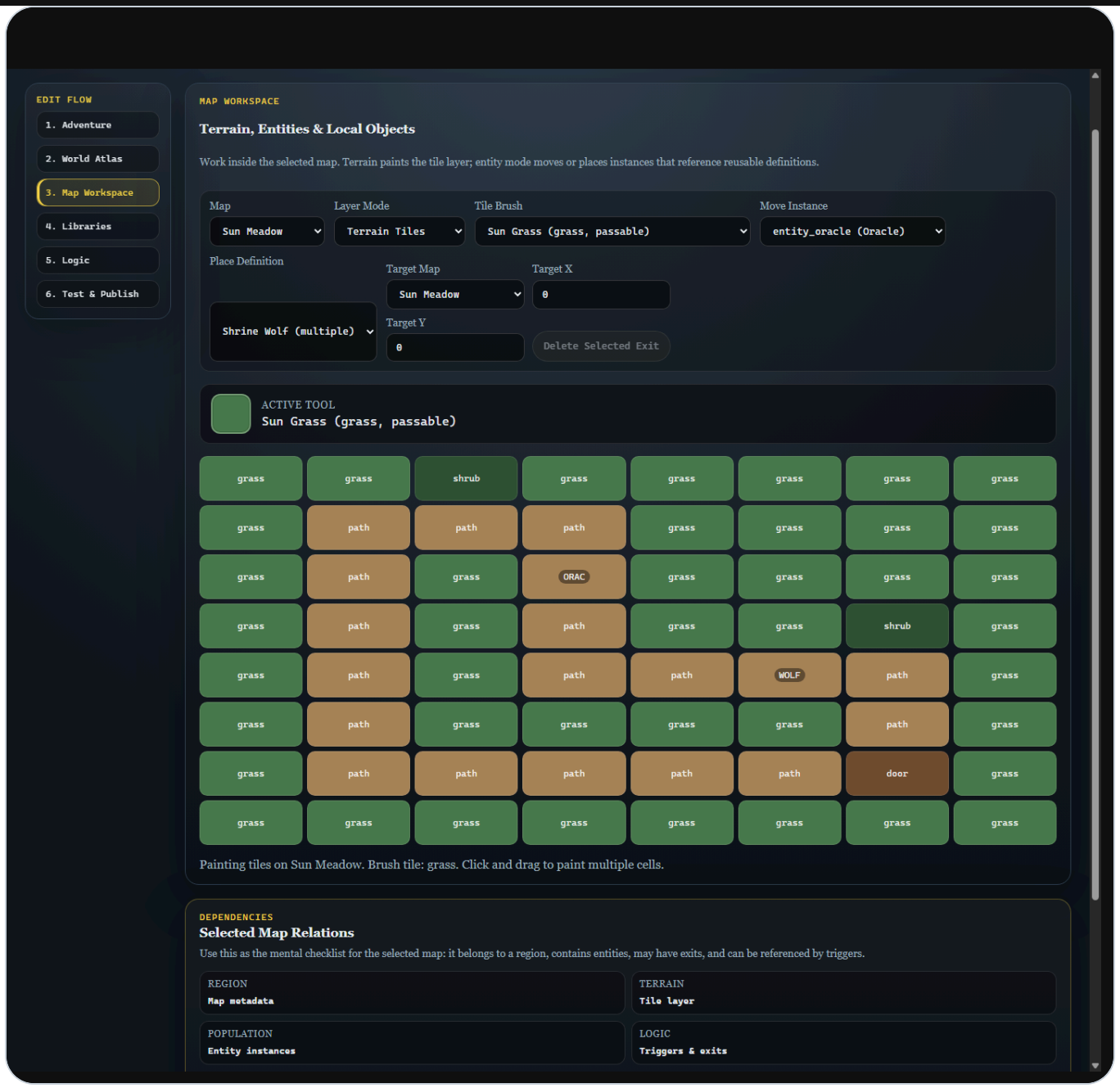
Try Adventuria-style variations: a fantasy shrine blessing, a sci-fi airlock repair, an urban spy handoff, or a supernatural hex-breaking ritual can all be represented with the same quest-stage data and trigger actions.

Latest Milestone Highlight: Focused Editor Workspaces

Milestone 18 changes the editor into a workspace switcher. Use the left `Edit Flow` navigation to choose one design area at a time. This keeps the screen from showing every possible data panel at once.



1. Click `World Atlas` to choose or create maps and review map/region relationships.
2. Click `Map Workspace` to paint terrain, place entities, or attach trigger markers on the selected map.
3. Click `Libraries` when you want reusable definitions such as entity templates and dialogue text.
4. Click `Logic` when you want the trigger builder, trigger markers, dialogue references, and rule details.
5. Click `Test & Publish` when you are ready to validate, save, publish, or open a release.



The important mental model is that the navigation does not create separate copies of the adventure. Every panel edits the same draft package; the editor is simply showing the slice of the construction set that matches your current design task.

4. Understand The Organized Editor Flow

Use the editor from left to right: Edit Flow navigation, Adventure Setup, World Atlas, Map Workspace, Dependencies, Libraries, Logic & Quests, and Test & Publish.

```
Adventure -> Regions -> Maps -> Tiles / Entities / Triggers
```

Reusable libraries sit beside that hierarchy because maps and triggers reference them.

5. Edit Metadata In Adventure Setup

1. Change the title to `Milestone 22 Adventuria Sampler`.
2. Update the description to mention map creation, tiles, entities, dialogue, triggers, and publishing.
3. Watch the validation summary update as the draft changes.

This changes adventure-wide metadata only. It does not change player position, maps, tiles, entities, triggers, or saves.

> **Latest milestone highlight: Milestone 18**
>
> Milestone 18 adds focused editor workspaces. Instead of seeing the entire construction set at once, you switch between Adventure, World Atlas, Map Workspace, Libraries, Logic, and Test & Publish so each task has a clearer home.

6. Edit World Structure

1. Select `Sun Meadow` as the current map.
2. Confirm its category is `local`.
3. Rename it to `Sun Meadow Test`.
4. Select `Inner Shrine` and confirm its category is `interior`.
5. Rename it to `Inner Shrine Test`.

`Region` is the larger organizing bucket. `Map` is the playable/editable space. `Map Category` describes map scale or purpose and can later drive filters, navigation, encounters, or renderers.

7. Create A Blank Map

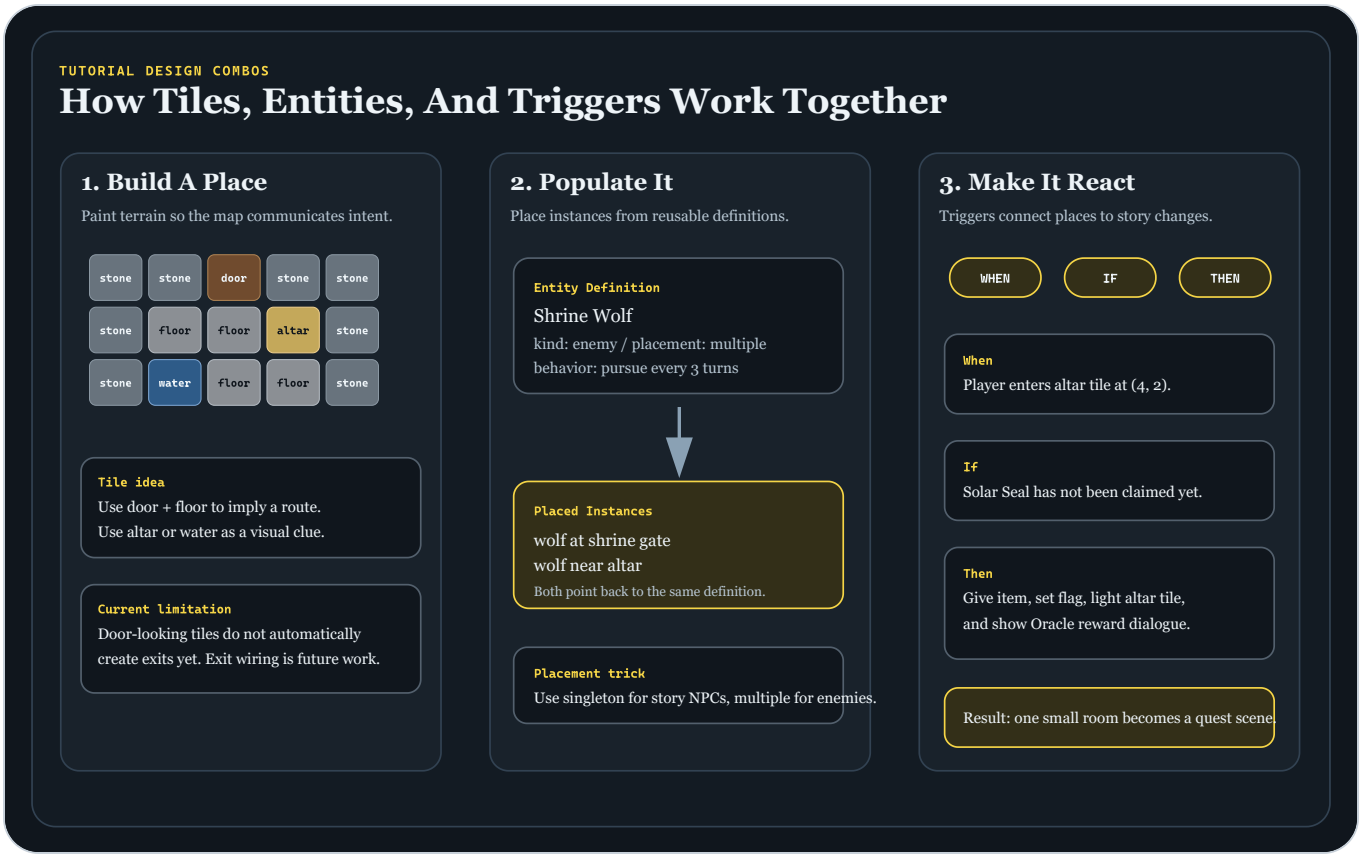
1. Name the map `Practice Dungeon`.
2. Set category to `dungeonFloor`.
3. Assign it to the Inner Shrine region if available.
4. Set width and height to `8`.
5. Set fill tile to `stone` or `floor`.
6. Click `Create Map`.

The new map is editable immediately. Milestone 20 adds exit/portal authoring so you can now wire authored maps together deliberately instead of waiting for hardcoded links.

8. Paint A Small Scene With Tiles

1. Set `Layer Mode` to `Terrain Tiles`.
2. Choose `floor`, confirm the `Active Tool`, and drag across several cells.
3. Choose `door` and paint one doorway tile.
4. Choose `altar` and paint one focal tile.
5. Choose `water`, `grass`, or `shrub` and paint a small visual clue.

Clever tile use: use door tiles as visual promises, altar tiles as natural trigger locations, water or shrub tiles as clues/barriers, and path/floor tiles to lead the player's eye.



9. Author A Door, Portal, Hatch, Or Gate

1. In `Map Workspace`, keep `Sun Meadow` selected.
2. Set `Layer Mode` to `Exits & Portals`.
3. Set `Target Map` to `Inner Shrine`, `Target X` to `1`, and `Target Y` to `4`.
4. Click the existing door at `(6, 6)`. The Active Tool should show an exit to Inner Shrine, and the Dependencies panel should list the edge.
5. Switch the workspace map to `Inner Shrine` and inspect the return exit to `Sun Meadow`.

This is the first map graph workflow: the grid is still local to one map, but the Dependencies panel shows the global consequences. If you make a one-way portal, the map graph will show only one edge; if you author a return exit, it becomes a two-way route.

Clever exit use: make a harmless-looking shrub a secret faerie portal, make an altar transport the player after a relic is claimed, or make a subway hatch connect a modern city to a mythic underworld. The logical operation is the same; the story and visual manifest sell the fantasy.

10. Move Existing Entities

1. Switch back to `Sun Meadow Test` or `Inner Shrine Test`.
2. Set `Layer Mode` to `Entity Instances`.

3. Pick an existing entity from `Move Instance`, such as the Oracle or Shrine Wolf.
4. Click a new destination cell and confirm the entity list updates.

You are moving an `EntityInstance`, not changing the reusable definition. The instance stores `definitionId`, `mapId`, `x`, and `y`.

> **Clever placement use:** put the Oracle near a path intersection, put a wolf near a reward route but not directly on an exit, and use spacing so turn cadence creates tension instead of a traffic jam.

11. Place A New Entity From A Definition

1. Choose `Shrine Wolf` from `Place Definition`.
2. Click `Place New`.
3. Click a destination cell.
4. Confirm the new wolf appears on the grid and in the map entity list.
5. Try choosing the Oracle definition if it is available.

The Oracle is `singleton`, so it should not be duplicated. The Shrine Wolf is `multiple`, so it can be placed repeatedly.

12. Highlight Milestones 15 And 19: Use Classified Libraries

MILESTONE 22

ACS Construction Set MVP

Back To Play | Save Draft | Reset Draft | Playtest Draft

EDIT FLOW

1. Adventure
2. World Atlas
3. Map Workspace
4. Libraries
5. Logic
6. Test & Publish

LIBRARIES

Game Library: Quests

Quests organize goals, stages, rewards, and source references so runtime objectives can grow beyond hardcoded demo text.

Library Focus

Quests

QUEST DEFINITIONS

- Claim the Solar Seal - stage count 4; Main Quests; rewards Solar Seal, Oracle blessing; Hear the Oracle, cross into the shrine, then return with the seal.

QUEST CATEGORIES

- Main Quests: Primary authored objective chains.

CREATE CATEGORY FOR CURRENT FOCUS

Category Name

Quest category name

Parent Category

No parent quest category

Description

Describe this quest category

Create Category

New categories will be created as quest categories.

QUEST DEFINITION EDITOR

Quests define objective text by stage. Runtime objective text reads the current quest stage instead of hardcoded page copy.

Quest

Claim the Solar Seal (4 stages)

Name

Claim the Solar Seal

Summary

Hear the Oracle, cross into the shrine, then return with the seal.

Stage Objective Text

```
Await the Oracle
Seek the shrine
Return to the Oracle
Quest complete
```

Reward Notes

Solar Seal, Oracle blessing

Source References

Milestone 5 sample quest

CREATE QUEST

New Quest Name

Recover the Moon Key

Create Quest Definition

Claim the Solar Seal starts at stage 0; 4 objective stage(s); 3 trigger reference(s).

1. Choose `Items` and confirm `Oracle Charm` and `Solar Seal` appear under `Item Definitions`. The Solar Seal is an item definition, not a hardcoded reward string.
2. Choose `Dialogue` and confirm `dialogue_intro`, `dialogue_shrine`, and `dialogue_return` appear under `Dialogue Definitions` in the `Oracle & Shrine Scenes` category.
3. While still on `Dialogue`, type a new category such as `Shrine Warnings`, add a description, and click `Create Category`. The new category is created as a dialogue category because the current focus is Dialogue.
4. Switch to `Skills`, `Flags`, `Quests`, `Traits`, `Spells`, `Tiles`, `Assets`, or `Custom` and notice the same category creator follows the selected kind of thing.
5. On `Tiles`, confirm terrain definitions such as `Grass`, `Path`, `Shrub`, `Stone`, `Altar`, `Door`, and `Water` appear as editable library records.

6. Return to `Entities`, select `Hero`, and notice the profile fields: `Life`, `Power`, `Speed`, `Skills`, and `Starting Possessions`.
7. Assign skills from the definition-backed skill list, then assign starter gear from the item-backed possessions list.
8. Select `Shrine Wolf`, rename it to `Trial Wolf`, keep `Placement` as `multiple`, and adjust behavior values such as detection range, leash range, or turn interval.

Profiles describe what a creature or character is. Behavior describes what it does each turn. Starting possessions describe what the party begins with. The Library Focus selector shows the object class you are editing, while the underlying package keeps items, dialogue, skills, flags, quests, and entities as separate reusable definitions with kind-specific categories.

> **Clever library use:** make the Solar Seal a quest relic, the Oracle a singleton entity with prophecy skills, the wolf a reusable enemy, and shrine triggers point to definitions instead of typed strings.

13. Edit Dialogue As A Library Object

1. Select the Oracle dialogue record.
2. Change the text to something recognizable, such as `The Oracle remembers this test.`
3. Keep the continue label readable.
4. Later, playtest and interact with the Oracle to confirm the changed text appears.

Dialogue is useful as feedback after a trigger changes a flag, grants an item, or changes a tile.

14. Highlight Milestone 16: Build A Trigger Without JSON

Use `Logic & Quests` as a tiny possibility sampler inspired by the original *Land of Adventuria* demo: a tutorial should hint at many genres and tricks, not just one plain room.

1. Select `trigger_shrine_reward`.
2. Keep `When` as `Enter Tile`, with the shrine altar coordinate and `Run Once` enabled.
3. In `If Conditions`, add `Flag Equals`: `quest_started` equals `true`.
4. In `Then Actions`, add `Show Dialogue`: `dialogue_shrine`.
5. Add `Give Item`: `Solar Seal`, quantity `1`.
6. Add `Set Flag`: `quest_stage` to `2`.
7. Add `Change Tile`: shrine altar coordinate to `altar-lit`.
8. Watch the advanced JSON mirror update from the builder output.

> **Adventuria-style variants:** a fantasy shrine can light an altar and grant a relic; a sci-fi transporter can teleport the player; an urban keypad can set an unlock flag and change a door tile; a spy dossier pickup can give an item, show a briefing, and set a clue flag.

The builder writes ordinary `TriggerDefinition.conditions` and `TriggerDefinition.actions`. The runtime does not care whether the rule came from JSON or

guided controls.

15. Highlight Milestone 17: Create And Place A Trigger Marker

1. In `Logic & Quests`, click `Create Trigger`.
2. Set `When: Type` to `Enter Tile`.
3. In `Map Workspace`, change `Layer Mode` to `Trigger Markers`.
4. Click a doorway, altar, water edge, or clue tile.
5. Confirm the cell shows a small trigger marker chip.
6. Return to `Logic & Quests` and confirm the map/x/y fields match the clicked cell.
7. Add a `Then` action such as dialogue, item reward, teleport, or tile change.
8. Read `Referenced Objects` to see every map, flag, dialogue, item, quest, and tile touched by the trigger.

> **Fun variants:** fantasy altar marker grants a relic; sci-fi floor marker teleports the player; urban keypad marker changes a locked door; spy dossier marker gives an item and sets a clue flag.

Use `Duplicate` to make a variation and `Delete` to remove scratch triggers. The runtime still receives ordinary trigger data.

16. Combine Tiles, Entities, And Triggers Into A Mini Scene

1. Paint a path or floor route toward a focal tile.
2. Put an `altar` or unusual tile at the destination.
3. Place a wolf near, but not directly on, the approach route.
4. Make sure the reward trigger points to the focal tile.
5. Use dialogue to explain what happened after the trigger fires.
6. Playtest and watch the event log, inventory, flags, and tile changes.

This is the heart of the construction set: a map cell can be terrain, a clue, a creature position, a trigger location, a reward moment, and a story beat.

17. Validate The Draft

1. Read the local validation summary.
2. Click `Validate Draft` in `Test & Publish`.
3. Confirm the API validation result appears.
4. Fix blocking errors before saving or publishing.

Common validation failures include out-of-bounds entities, tile layers with the wrong tile count, missing tile definitions, broken references, or duplicate singleton entities.

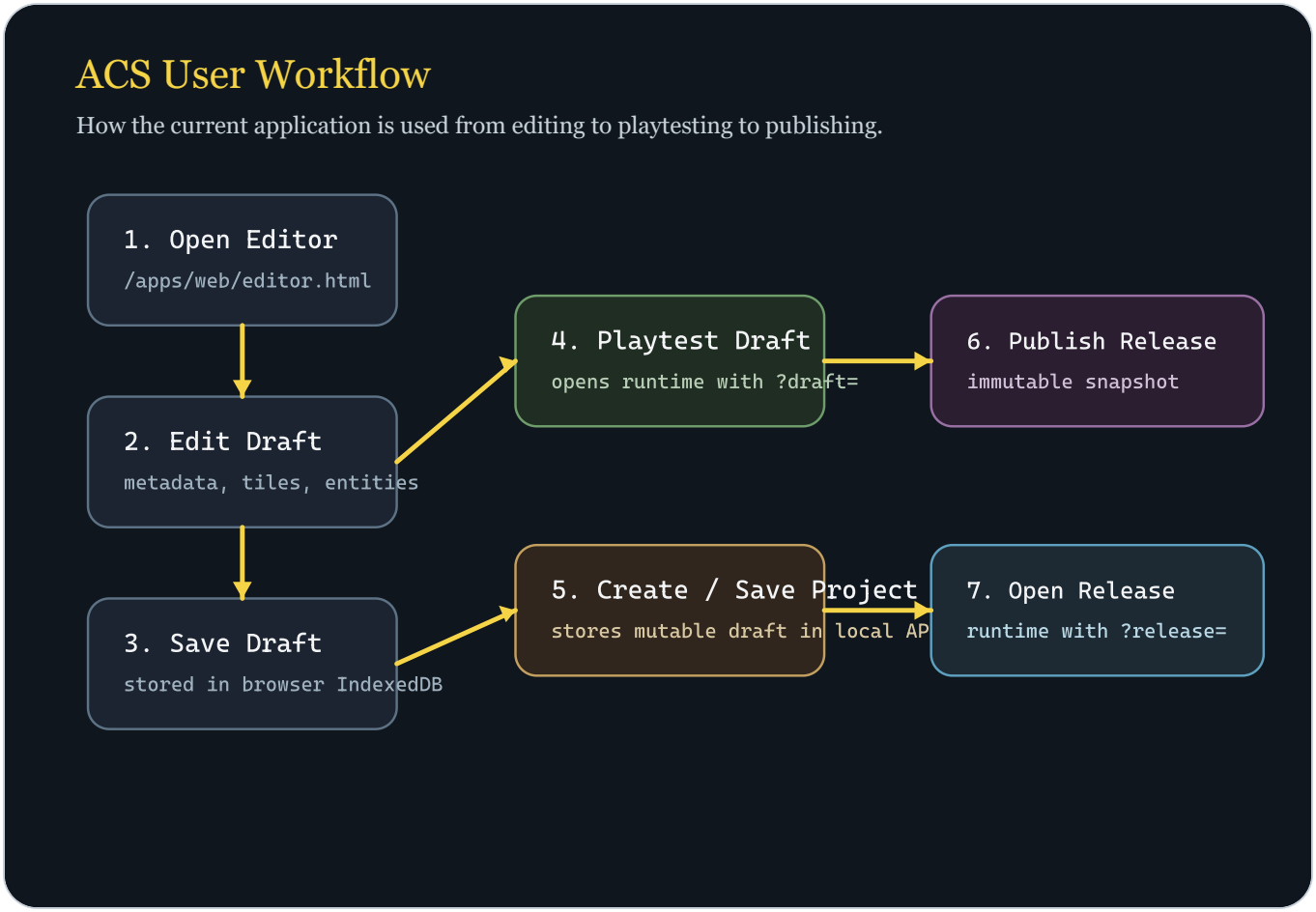
18. Save And Playtest The Draft

1. Click `Save Draft`.
2. Click `Playtest Draft`.

3. Confirm the edited title/source is loaded as a draft playtest.
4. Walk to the Oracle and confirm dialogue edits.
5. Visit the shrine and confirm reachable tile or trigger edits.
6. Use `Save`, `Load`, and `Reset` in the playtest session.

A newly created blank map must be connected through the Exits & Portals layer before it is reachable through gameplay. Edits to existing reachable maps are easiest to verify right now.

19. Create, Save, Publish, And Open A Release

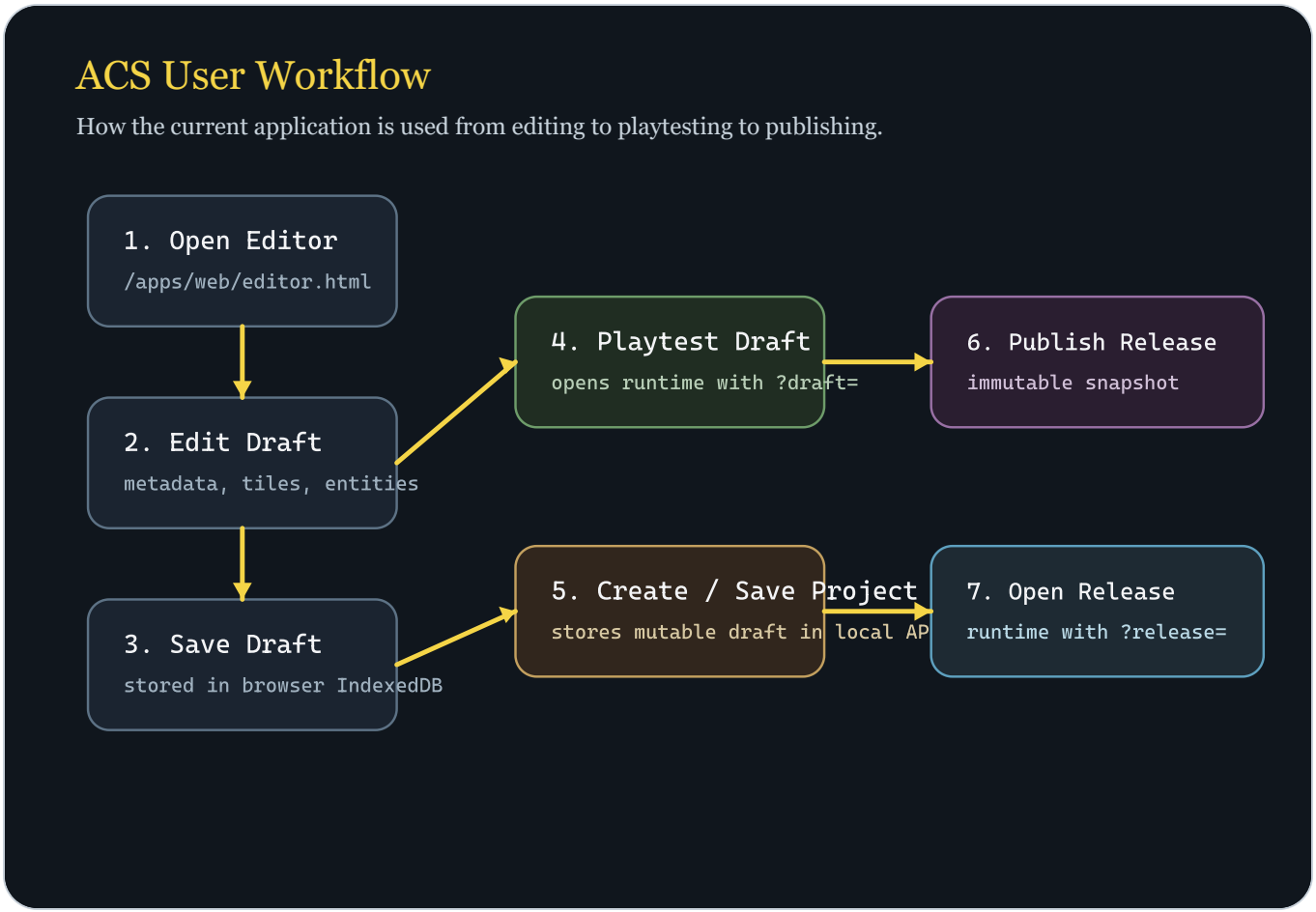


1. Return to the editor.
2. Click `Create Project` if no backend project exists yet.
3. Click `Save Project` to send the current draft to the API.
4. Click `Publish Release` to create an immutable release snapshot.
5. Click `Open Latest Release`.
6. Confirm the release loads separately from the local draft playtest.

20. Reset Safely When Finished

Click `Reset Draft` if you want the editor to return to the built-in sample adventure. This removes the browser-local draft but does not delete backend projects or published releases.

Projects And Published Releases



The editor can move a draft through five project stages:

1. **Validate Draft**: run the backend validation report without publishing.
2. **Create Project**: create a mutable backend project from the current draft.
3. **Save Project**: update the mutable backend draft.
4. **Publish Release**: create an immutable release snapshot.
5. **Open Latest Release**: launch that published release in the runtime.

Important distinction

- `Save Draft` writes to browser storage.
- `Save Project` writes to the local API.
- `Publish Release` freezes a release snapshot rather than editing it in place.

Where Data Lives

Browser storage

- Runtime saves
- Local drafts
- Remembered active project id

Local API storage

- Mutable project drafts
- Immutable published releases
- Stored locally in `apps/api/data/store.json`

Current Limitations

- No real user accounts yet
- No cloud backend yet
- No asset upload flow yet
- No deletion of maps or automatic exit/portal wiring yet
- No deletion of entity instances in the editor yet
- Trigger records can now be created, duplicated, deleted, edited, and placed as map markers.
- Brand-new dialogue/item/quest record creation remains future work.
- The classic visual mode uses manifest-driven procedural sprites; Milestone 24 is planned to add a true built-in pixel-art editor, stocked fantasy/science-fiction/modern-spy/superhero/science-fantasy/supernatural-investigation/urban-fantasy starter libraries, splash-screen selection, starting music selection, richer sprite sheets, animations, and higher-resolution asset-pack preparation.

Documentation Generation Instructions

- The User Guide tutorial must exercise every feature currently available in the application.
- The newest milestone's features must be highlighted explicitly near the start of the tutorial.
- The User Guide PDF must include current runtime/editor screenshots or screenshot-style graphics.
- The System Reference must deeply explain how major features are implemented with end-to-end flows.
- Mermaid diagrams in Markdown need readable rendered equivalents in HTML/PDF.
- Diagrams, screenshots, code blocks, and callout boxes should avoid page splits wherever practical.
- If screenshot button text spills outside a button, regenerate the graphic with smaller text before publishing.

Troubleshooting

The editor says the API is unavailable

```
node .\apps\api\dist\index.js
```

Validate Draft fails or publish stays disabled

- A trigger may reference a missing map, item, dialogue, or quest.
- An entity or start position may be outside a map's bounds.
- A singleton entity definition may have more than one placed instance.
- A tile layer may have the wrong tile count for its dimensions.

A published release will not open

- The API may not be running.
- The release id may not exist anymore.
- The local API store may have been cleared.

My draft changes are gone

- Use `Save Draft` for browser-local storage.
- Use `Save Project` for backend storage.

Playtest Draft opens the sample adventure instead

The runtime falls back to the built-in sample when it cannot find the draft key passed by the editor.